

Ghidul primului program C in Linux

1. Scopul lucrării

Lucrarea de față este un ghid pentru scrierea, compilarea și executia primului program în C în sistemul de operare Linux.

2. Scrierea programului

Se va folosi pentru scrierea programului editorul `mcedit`, dar orice editor text poate fi folosit (`gedit`, `vi`, `nano`, etc.). Pentru **`mcedit`** (parte a pachetului de programe `midnight commander`) se va deschide fișierul `laborator1.c` astfel:

`mcedit laborator1.c`

Se scrie în continuare programul:

```
/* program C pentru afisarea unui mesaj */  
  
main(){  
  
    printf("Acesta este primul program in C\n");  
  
}
```

Pentru a salva codul scris se apasă tasta `F2` iar pentru a ieși se apasă tasta `F10`.

Elementul constructiv al unui program C este funcția. O funcție este o secțiune de program, identificată printr-un nume și parametrizată, construită folosind declarații, definiții și instrucțiuni de prelucrare. Atunci când este apelată, funcția calculează un anumit rezultat sau realizează un anumit efect. **O funcție în C este urmată de paranteze între care se introduc parametrii funcției. Paranteze se păstrează chiar dacă funcția nu are parametri ca în acest caz: `main()`.**

Un program este constituit dintr-o mulțime de funcții, declarate pe un singur nivel (fără a se imbrica unele în altele), grupate în module program.

Funcția `main()` este prezentă în orice program C. Execuția programului începe cu `main()`. Funcția `main()` poate întoarce un rezultat întreg (`int`) sau nici un rezultat (`void`). Numai în C este posibil să nu specificăm tipul rezultatului întors de funcție, acesta fiind considerat în mod implicit `int`.

Programul folosește un comentariu, delimitat prin `/*` și `*/` care, prin explicații în limbaj natural, crește claritatea programului. Comentariul este constituit dintr-o linie sau mai multe linii, sau poate apărea în interiorul unei linii. Nu se pot include comentarii în interiorul altor comentarii.

Alta modalitate de introducere a comentariilor este în urma simbolului `//`

Compiarea programului

Compilerul este gcc (GNU Compiler Collection) este acela care transformă codul sursă scris anterior în cod mașină.

Pentru a compila, se folosește:

```
gcc laborator1.c -o laborator1
```

În exemplul de mai sus:

- **laborator1.c** este fișierul conținând codul sursă
- **laborator1** este fișierul binar (executabil). Se recomandă să nu folosiți extensie pentru programele binare.

Execuția programului

Programul binar se execută cu:

```
./laborator1
```

Include

Într-un program este posibil ca într-un program să avem nevoie de funcții suplimentare. Linia **#include <unistd.h>** de exemplu anunță compilatorul că trebuie să insereze fișierul antet `unistd.h`. Fișierele antet au prin convenție extensia `.h`. Fișierul de inclus este căutat într-o zonă standard de includere, în care sunt memorate fișierele antet ale compilatorului C, dacă numele este încadrat între paranteze unghiulare (`<` și `>`), sau căutarea se face în zona curentă de lucru, dacă fișierul este încadrat între ghilimele(`"`). Fișierele antet sunt foarte utile în cazul funcțiilor standard de bibliotecă; fiecare categorie de funcție standard are propriul fișier antet

1.2. Câteva elemente necesare scrierii unor programe C/C++ foarte simple.

1.2.1. Directiva `define`.

Directiva `#define` nume text este o macrodefiniție. Prelucrarea acesteia, numită macroexpandare, înlocuiește fiecare apariție a numelui prin textul asociat.

O aplicație o reprezintă creerea de constante simbolice. De exemplu:

```
#define PI 3.14159
```

```
#define mesaj "Bonjour madame"
```

```
#define MAX 100
```

O constantă simbolică astfel definită nu poate fi redefinită prin atribuire.

1.2.2. Tipuri

Fiecărui nume i se asociază un tip, care determină ce operații se pot aplica acelui nume și cum sunt interpretate acestea. De exemplu:

`char c='a';` // c este o variabilă caracter inițializată cu 'a'

`int f(double);` // f este o funcție de argument real cu rezultat întreg

1.2.3. Definiții și declarații de variabile

O valoare constantă se reprezintă textual (este un literal) sau printr-un nume - constantă simbolică. O variabilă este un nume (identificator) care desemnează o locație de memorie în care se păstrează o valoare.

O variabilă se caracterizează așadar prin: nume (adresă), tip și valoare, atributul valoare putând fi modificat. De exemplu:

`int n, p;`

`char c;`

`float eps;`

O variabilă poate fi inițializată la declararea ei. De exemplu:

`float eps=1.0e-6;`

Inițializarea se face numai o dată, înaintea execuției programului.

Variabilele externe și statice sunt inițializate implicit la 0.

Pentru o variabilă automatică (declarată în interiorul unui bloc), pentru care există inițializare explicită, aceasta este realizată la fiecare intrare în blocul care o conține.

1.2.4. Atribuirea.

Atribuirea simplă este de forma: `variabilă = expresie` și are ca efect modificarea valorii unei variabile.

Operatorii de incrementare folosiți în atribuiri au efecte diferite. Astfel:

`a = ++b` este echivalentă cu `b=b+1; a=b;` în timp ce:

`a = b++` are ca efect `a=b; b=b+1;`

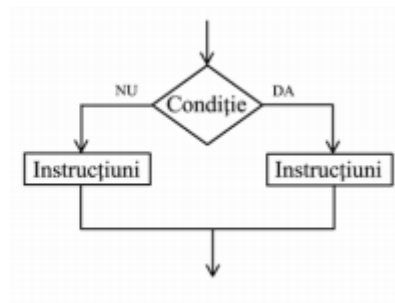
Instrucțiuni C

- Instrucțiunea vidă ;
- Instrucțiunea de atribuire `a = 3`
- Instrucțiunea compusă (bloc)

`{ instr1; instr2; etc }`

- Instrucțiunea if

`if (expresie) Instrucțiune1;
else Instrucțiune2`



Instrucțiunea *if*

■ Exemple:

```
if(x)
    if(y) printf("1");
    else printf("2");
```

```
if(x){
    if (y) printf("1");
}
else printf("2");
```

```
If (i=0) printf("Variabila i are valoarea 0");
else printf("Variabila i are o valoare diferita de 0");
```

Instrucțiunea *switch*

```
switch (expresie) {
    case c1: instr1;
    [case c2: instr2;]
    ...
    [default: instr_default;]
}
```

- c1, c2, etc – constante sau expresii întregi (inclusiv char)

Exemplu:

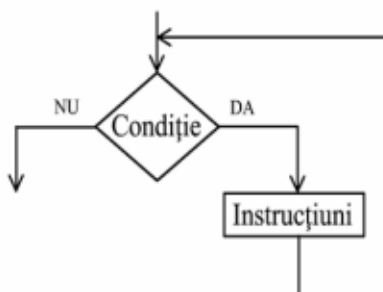
Instrucțiunea *switch*

```
char c; // c poate fi +,-,*,/  
switch ( c=getchar()) {  
    case ' + ': r =a + b; break;  
    case ' - ': r =a - b; break;  
    case ' * ': r =a * b; break;  
    case ' / ': r =a / b; break;  
    default: printf("Eroare!\n");  
}
```

Instrucțiunea *while*

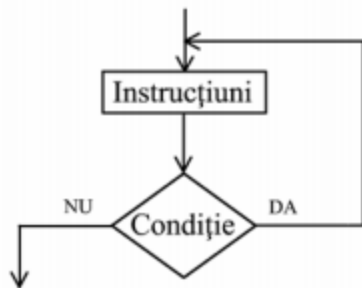
- while (expresie)
 instrucțiune;

- Exemplu:
#include<stdio.h>
void main(void)
{
 int a=18, b=12, r;
 while (a%b>0) {
 r=a%b;
 a=b;
 b=r;
 }
 printf("cmmdc este %d\n",b);
}



Instrucțiunea do while

```
do  
    instrucțiune;  
while(expresie);
```

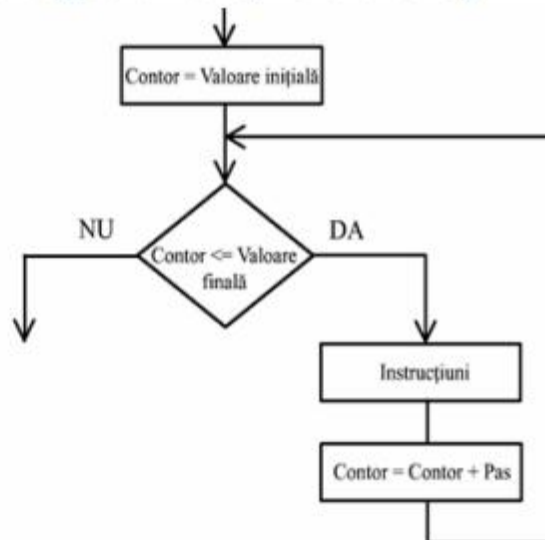


Exemplu:

```
#include<stdio.h>  
void main(void)  
{  
    int s=0,i=0,n;  
    printf("introduceti n:\n");  
    scanf("%d",&n);  
    do{  
        s+=i;  
        i++;  
    }while (i<=n);  
    printf("suma primelor %d nr. nat. este  
           %d\n",n,s);  
}
```

Instrucțiunea for

```
for (conditie_start; conditie_continuare; re-evaluare)  
    instrucțiune;
```



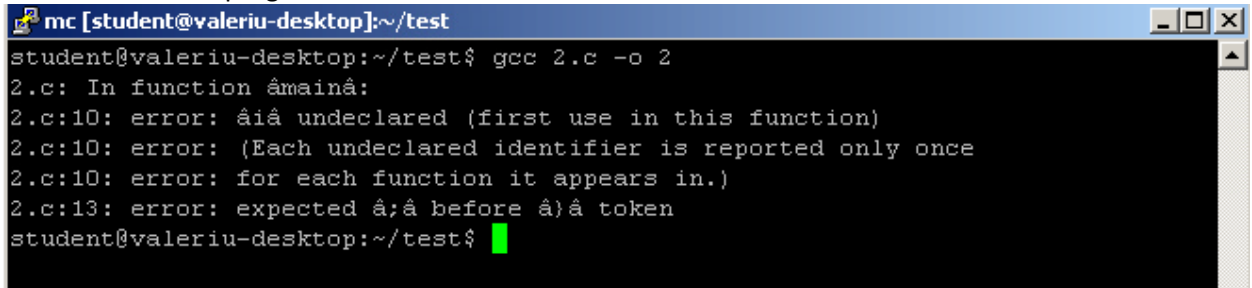
Instrucțiunea for

■ Exemplu:

```
#include<stdio.h>
void main(void)
{
    int n, fact=1, i;
    printf("introduceti n: ");
    scanf("%d", &n);
    for(i=1; i<=n; i++) fact*=i;
    printf("%d! = %d\n",n,fact);
}
```

3. Desfasurarea laboratorului

- Se va studia programul model, apoi se va scrie cu un editor de text (mcedit, nano, gedit, etc.), se va compila si se va executa.
- Se vor recapitula noțiunile de baza privind programarea prezentate in partea a doua a laboratorului.
- Se vor scrie programele si se vor executa. Se va urmări rezultatul execuției. Daca apar erori se va identifica linia la care a apărut eroarea si se va corecta. De exemplu codul de mai jos indica o eroare la linia 10 a programului.



```
mc [student@valeriu-desktop]:~/test
student@valeriu-desktop:~/test$ gcc 2.c -o 2
2.c: In function âmainâ:
2.c:10: error: âiâ undeclared (first use in this function)
2.c:10: error: (Each undeclared identifier is reported only once
2.c:10: error: for each function it appears in.)
2.c:13: error: expected â;â before â}â token
student@valeriu-desktop:~/test$
```